DAY 08 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

NFTs: ERC-721 and ERC-1155

Thursday 22 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-08

Learning objectives

1. Explain an NFT as an ownership table (`tokenId → owner`) plus a metadata link.
2. Use the two permission types correctly: `approve` (one token) and `setApprovalForAll` (everything).
3. Know why `safeTransferFrom` exists, and why its receiver call is also a re-entrancy door.
4. Build a paid collection: supply cap, price, per-wallet limit, metadata, royalties (ERC-2981).
5. Use ERC-1155 for many item types (coins and one-of-ones) in one contract, with batches.
6. Test a standard, not a contract: one suite and a differential fuzz test against OpenZeppelin.

The concept, step by step

Step 1 — An NFT is a table

Day 07's ERC-20 stored `address → amount`. ERC-721 stores `tokenId → owner`: every row is one numbered item with exactly one owner. "Ram owns NFT #7" simply means `ownerOf(7) == Ram`. A token that was never minted (or was burned) has no owner, and `ownerOf` **reverts** rather than answering "nobody".

ERC-20 (Day 07)

```
address → amount
Ram → 500
Sita → 120
```

ERC-721 (today)

```
tokenId → owner
#1 → Ram
#2 → Ram
#3 → Sita
```

ERC-1155 (today)

```
(id, owner) → amount
(GOLD, Ram) → 1000
(SWORD, Ram) → 5
(CROWN, Ram) → 1
```

Figure 1 — Three token standards, three tables. The shape of the table is the whole difference.

Step 2 — The picture is not on the chain

`tokenURI(7)` returns a **link** to a JSON file (name, description, image link). The chain proves *who owns #7*; the JSON says *what* #7 is. An `ipfs://` link is a content hash, so the file cannot be swapped silently; an `https://` link can. That is why today's evening improvement adds `freezeMetadata()`.

Step 3 — Two kinds of permission

`approve(Hari, 7)` lets Hari move token #7 once; it is wiped automatically when #7 moves. `setApprovalForAll(market, true)` lets an operator move **every** token you own, including ones you receive later. Marketplaces need it, and phishing sites abuse it: one signature can empty a wallet.

Step 4 — Safe transfers, and the door they open

Plain `transferFrom` into a contract that does not understand NFTs locks the token there forever. `safeTransferFrom` first calls `onERC721Received` on the receiver and refuses unless it returns its own 4-byte selector. But that call hands control to the receiver *in the middle of your function*:

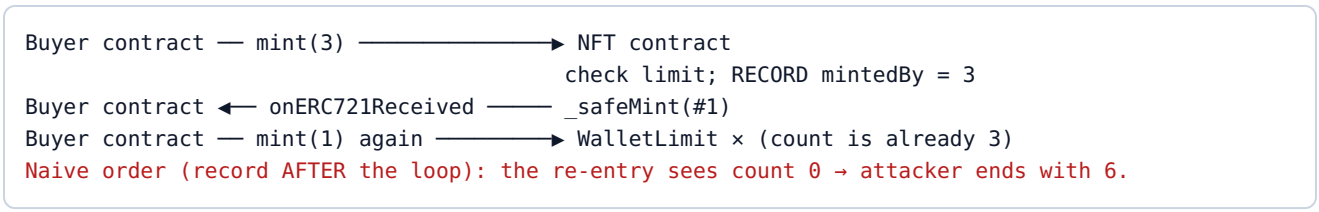


Figure 2 — Re-entrant minting. Same lesson as Day 06: checks → effects → interactions.

Step 5 — Royalties are a question, not a rule

ERC-2981 adds `royaltyInfo(id, salePrice)`, which answers "who should get how much". With 500 basis points (5%), a 2 ETH sale returns 0.1 ETH. Nothing in the token forces a marketplace to pay; a plain transfer pays nothing. Integer maths rounds down: 19 wei × 5% = 0.

Step 6 — ERC-1155: many items, one contract

Each id has its own supply. GOLD (cap 1,000,000) behaves like a coin, SWORD (cap 100) like a limited edition, CROWN (cap 1) like an NFT. `mintBatch` and `safeBatchTransferFrom` move several types in one transaction. There is no unsafe transfer at all, and there is **no per-item approval**: approving a marketplace for GOLD also lets it move the CROWN.

Key code walkthrough

MiniNFT.transferFrom — permission first, then the books

```
address owner = ownerOf(tokenId);            // reverts if the token does not exist
if (msg.sender != owner && _tokenApprovals[tokenId] != msg.sender
    && !isApprovedForAll(owner, msg.sender))
    revert ERC721InsufficientApproval(msg.sender, tokenId);
if (owner != from) revert ERC721IncorrectOwner(from, tokenId, owner); // order fixed (M1)
delete _tokenApprovals[tokenId];              // the old approval must not survive the sale
unchecked { _balances[from] -= 1; _balances[to] += 1; }
_owners[tokenId] = to;
emit Transfer(from, to, tokenId);
```

MiniNFT._checkOnERC721Received — ask before sending

```
if (to.code.length == 0) return;                // a normal wallet can always hold NFTs
try IERC721Receiver(to).onERC721Received(msg.sender, from, tokenId, data) returns (bytes4 r) {
    if (r != IERC721Receiver.onERC721Received.selector) revert ERC721InvalidReceiver(to);
} catch (bytes memory reason) {
    if (reason.length == 0) revert ERC721InvalidReceiver(to); // no hook at all
    assembly { revert(add(32, reason), mload(reason)) }      // pass the receiver's own error on
}
```

LearningNFT.mint — record before you hand over control

```
// checks
if (quantity == 0 || quantity > MAX_PER_WALLET) revert InvalidQuantity(quantity);
if (quantity > MAX_SUPPLY - totalMinted) revert SoldOut(...);
if (mintedBy[msg.sender] + quantity > MAX_PER_WALLET) revert WalletLimit(...);
if (msg.value != MINT_PRICE * quantity) revert WrongPayment(...);   // exact: no silent overpay
// effects  (BEFORE the receiver runs)
uint256 firstId = totalMinted + 1;  totalMinted += quantity;  mintedBy[msg.sender] += quantity;
// interactions
for (...) _safeMint(msg.sender, firstId + i);   // may call msg.sender back
```

GameItems.mintBatch — the same id twice must not dodge the cap

```
for (i ...) { already = 0;
    for (j < i) if (ids[j] == ids[i]) already += amounts[j];   // running total per id
    _checkCap(ids[i], already + amounts[i]); }
_mintBatch(to, ids, amounts, "");      // [CROWN, CROWN] × [1, 1] → ExceedsMaxSupply(2, 1, 1)
```

Evening fix and improvement (LearningNFT)

```
function renounceOwnership() public override onlyOwner {
    uint256 held = address(this).balance;
    if (held != 0) revert FundsStillHeld(held);    // E1: otherwise the ETH is locked forever
    super.renounceOwnership(); }
function freezeMetadata() external onlyOwner { ... metadataFrozen = true; }  // one-way
// setBaseURI now also emits EIP-4906 BatchMetadataUpdate(1, 2^256-1) so marketplaces refresh
```

Results from the run

What	Result
Gas: mint (MiniNFT vs OpenZeppelin)	68,560 vs 68,807
Gas: transferFrom (MiniNFT vs OpenZeppelin)	54,629 vs 55,077
Differential fuzz test (120 random programs)	1,394 steps: 454 succeeded on both, 940 failed on both with the identical error
Re-entrant mint, limit 3	Naive contract: attacker holds **6**. LearningNFT: **3** (3 re-entries tried, all refused)
Royalty on a 2 ETH sale at 500 bps	0.1 ETH to the artist
ERC-1155: 3 item types	One batch 201,004 gas vs 3 single mints 266,127 gas (−24%)
Merkle allowlist (exercise 7)	Writing 1,000 addresses: 22,858,999 gas. Whole Merkle contract: 1,237,697 gas (18.5×)
Quality	117/117 tests · ESLint + Solhint clean · npm audit 0 · Slither: 5 informational (explained) · mutation 38/38 caught

QA & security findings

ID	Sev.	Found by	Finding	Fix
E1	Medium	Evening audit	Owner could `renounceOwnership()` with mint income inside; nobody could ever withdraw it again.	Renounce reverts with `FundsStillHeld` while the balance is not 0.
M1	Low	Shared test suite	MiniNFT checked `from` before permission, so a stranger got a different error than from OpenZeppelin.	Permission checked first.
S1	Low	Slither	Two locals relied on implicit zero.	Initialised explicitly.
L1	Info	Solhint	3 lines of assembly to re-throw the receiver's error.	Kept and justified.

Also attacked and safe: buying from inside the owner's giveaway mint (fresh ids, cap kept), read-only re-entrancy (inside the hook the new owner is already recorded and the old approval is gone), the last-token race (loser refused, keeps their

ETH), royalty receiver 0x0, mint to 0x0, empty ERC-1155 batch. Accepted: royalties are not enforced, operator approval is all-or-nothing, metadata is a link (now freezable), $O(n^2)$ duplicate check in owner-only batches.

Common mistakes

- Updating limits or counters **after** `_safeMint` (re-entrant mint).
- Using `transferFrom` to send to a contract: the token can be stuck forever.
- Accepting `msg.value >= price`: buyers silently overpay and the extra is kept.
- Treating ERC-2981 as enforcement. It is a polite request.
- Storing an `https://` metadata link that the owner can change at any time.
- Clicking "approve all" on an unknown site: the operator gets every NFT you own.
- Letting the owner renounce with funds inside (today's E1).

Five interview questions

1. How is ERC-721 different from ERC-20?
ERC-20 maps an address to an amount of identical units; ERC-721 maps each token id to one owner, so every unit is distinct. ERC-721 also adds per-token approval and safe transfers.

2. Why does `safeTransferFrom` exist, and what risk does it add?
It refuses to send to contracts that cannot handle NFTs, by calling `onERC721Received`. That call gives the receiver control mid-function, so state must be final before it (re-entrancy).

3. What does `supportsInterface(0x80ac58cd)` mean?
ERC-165: "do you implement ERC-721?". The id is the XOR of the 9 ERC-721 function selectors.

4. Are ERC-2981 royalties guaranteed?
No. The contract only reports the amount; the marketplace decides whether to pay. Enforcing it means restricting transfers.

5. When would you choose ERC-1155?
When one project has many token types, some fungible and some unique (game items, tickets with tiers), and wants batch transfers and one deployment.

Exercises summary

Level	Exercise	Key idea
Easy	Rebuild ownership from `Transfer` events; compute the interface id; what `tokenURI` stores	Events are the history; `0x80ac58cd`; a link, not the image
Medium	Soulbound certificate; fix the re-entrant mint	Block transfers in `_update`; record before `_safeMint`
Challenge	Merkle allowlist presale; measure the storage alternative	One 32-byte root, 10-hash proofs, 18.5× cheaper than writing the list

Cheat sheet

ERC-721: `ownerOf`, `balanceOf`, `transferFrom`, `safeTransferFrom` ×2, `approve`, `getApproved`, `setApprovalForAll`, `isApprovedForAll`. Events: `Transfer`, `Approval`, `ApprovalForAll`.
Mint / burn = `Transfer` from / to `0x0`.
Interface ids: ERC-165 `0x01ffc9a7` · ERC-721 `0x80ac58cd` · Metadata `0x5b5e139f` · ERC-2981 `0x2a55205a` · ERC-1155 `0xd9b67a26` · EIP-4906 `0x49064906`.
Receiver hook: return `onERC721Received.selector` (`0x150b7a02`).
Royalty = salePrice × bps / 10,000, rounded down.
ERC-1155: `balanceOf(owner, id)`, `balanceOfBatch`, `safeBatchTransferFrom`, URI template `{id}`, operator approval only.
Rule of the day: any external call (including `_safeMint`) is a door. Check, record, then call.

References

1. EIP-721: Non-Fungible Token Standard — https://eips.ethereum.org/EIPS/eip-721
2. EIP-1155: Multi Token Standard — https://eips.ethereum.org/EIPS/eip-1155
3. EIP-2981: NFT Royalty Standard — https://eips.ethereum.org/EIPS/eip-2981
4. EIP-165: Standard Interface Detection — https://eips.ethereum.org/EIPS/eip-165
5. EIP-4906: Metadata Update Extension — https://eips.ethereum.org/EIPS/eip-4906
6. OpenZeppelin Contracts 5.x: ERC-721 — https://docs.openzeppelin.com/contracts/5.x/erc721
7. OpenZeppelin Contracts 5.x: ERC-1155 — https://docs.openzeppelin.com/contracts/5.x/erc1155
8. ethereum.org: Non-fungible tokens — https://ethereum.org/en/nft/
9. Solidity docs: Security considerations (re-entrancy) — https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
10. Antonopoulos & Wood, *Mastering Ethereum*, ch. 10 (Tokens) — https://github.com/ethereumbook/ethereumbook